AegisFlow

DevSecOps Pipeline Orchestrator Agent

Super-technical architecture, functionality map, and implementation overview

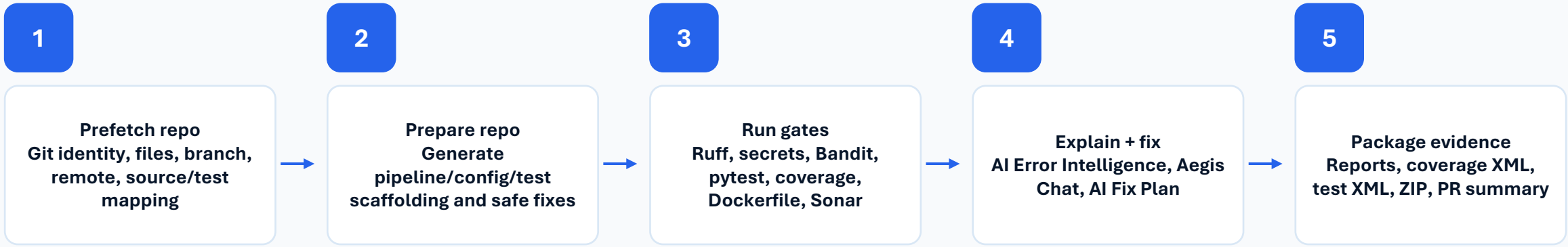
CI/CD cockpit | Security gates | Coverage evidence | AI explanations | SonarQube quality gate | AI Fix Plan | Downloadable evidence pack

Local repository → PR validation → CI/CD checks → SonarQube → evidence → governance → deployment readiness

Designed for Python Azure Function DevSecOps standardization and validation evidence.

What AegisFlow does

A local-first agentic cockpit that turns a repository into PR-ready DevSecOps evidence.



Primary outcome

Developers know whether the repo can be safely raised for PR.
Reviewers receive evidence instead of scattered logs.
Risky changes require approval and human-in-the-loop decisions.

Why it matters

Shortens feedback loops before Azure DevOps pipeline execution.
Makes acceptance criteria visible and auditable.
Supports a central template CI/CD model across function repos.

Functionality inventory

Every capability implemented or designed into the orchestrator.

Domain	Capabilities
Repo intelligence	Detect Git root, branch, remote, provider, changed files, source files, test mapping, protected folders
Repo restructuring	Create/update pipeline files, requirements-dev, pytest.ini, .coveragerc, Sonar config, .gitignore entries
Validation gates	Compile check, Ruff format/lint, secret scan, detect-secrets, Bandit, Pytest, coverage, Hadolint
SonarQube	Scan configuration, scanner bootstrap, coverage upload, quality gate wait, skipped/failed/pass visibility
AI layer	Ollama bootstrap, qwen2.5-coder support, deterministic fallback, Aegis Chat, error explanations
AI Fix Plan	Exact diffs, approval checkbox, patch application, rerun affected validation, before/after result
Evidence	Markdown/JSON reports, coverage.xml, test-results.xml, evidence ZIP, Azure DevOps-style coverage view
Governance	Acceptance criteria, release decision support, human-in-loop approvals, safety gates, PR comment

How AegisFlow fits into the target DevSecOps model

It does not replace Azure DevOps; it prepares, validates, explains, and packages evidence before the cloud pipeline.

AegisFlow responsibilities

Local evidence generation
Failure diagnosis
Safe fix planning
Acceptance criteria readiness
PR comment generation

Azure DevOps responsibilities

Branch policies and PR approval
Cloud pipeline run
Published test / code coverage results
Artifact retention
Deployment permissions

Human ownership

Merge approval
Production release
Security exceptions
Architecture sign-off
Compliance sign-off

Super-technical system architecture

Layered view of UI, orchestration, validation engines, integrations, and evidence outputs.

User Interface Layer

Streamlit dashboard: Repo Preflight, Acceptance Criteria, Live Progress, Report & Downloads, SonarQube, AI Chat, Governance, PR Comment

↓

Agent Orchestration Layer

AegisFlow Orchestrator Agent: run plan, state, protected paths, safety gates, command execution, session persistence

↓

Validation & Intelligence Layer

Repo inspector, file generator, validation runner, failure classifier, AI fix planner, evidence generator, governance engine

↓

External Integration Layer

Git, Azure DevOps API, SonarScanner, Ollama/qwen2.5-coder, Hadolint, Pytest, Ruff, Bandit, detect-secrets

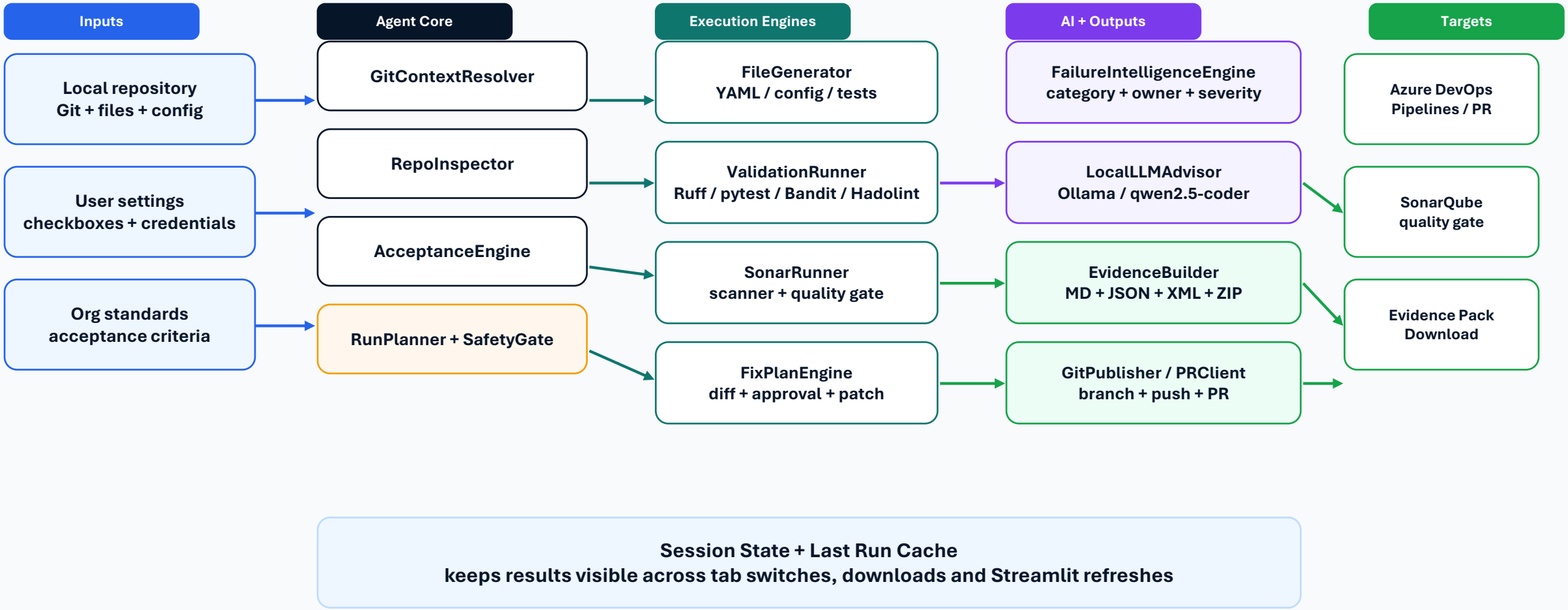
↓

Target Platform Layer

Azure Repos/GitHub, Azure Pipelines, SonarQube VM/SonarCloud, Azure Function App, Azure Artifacts, Key Vault

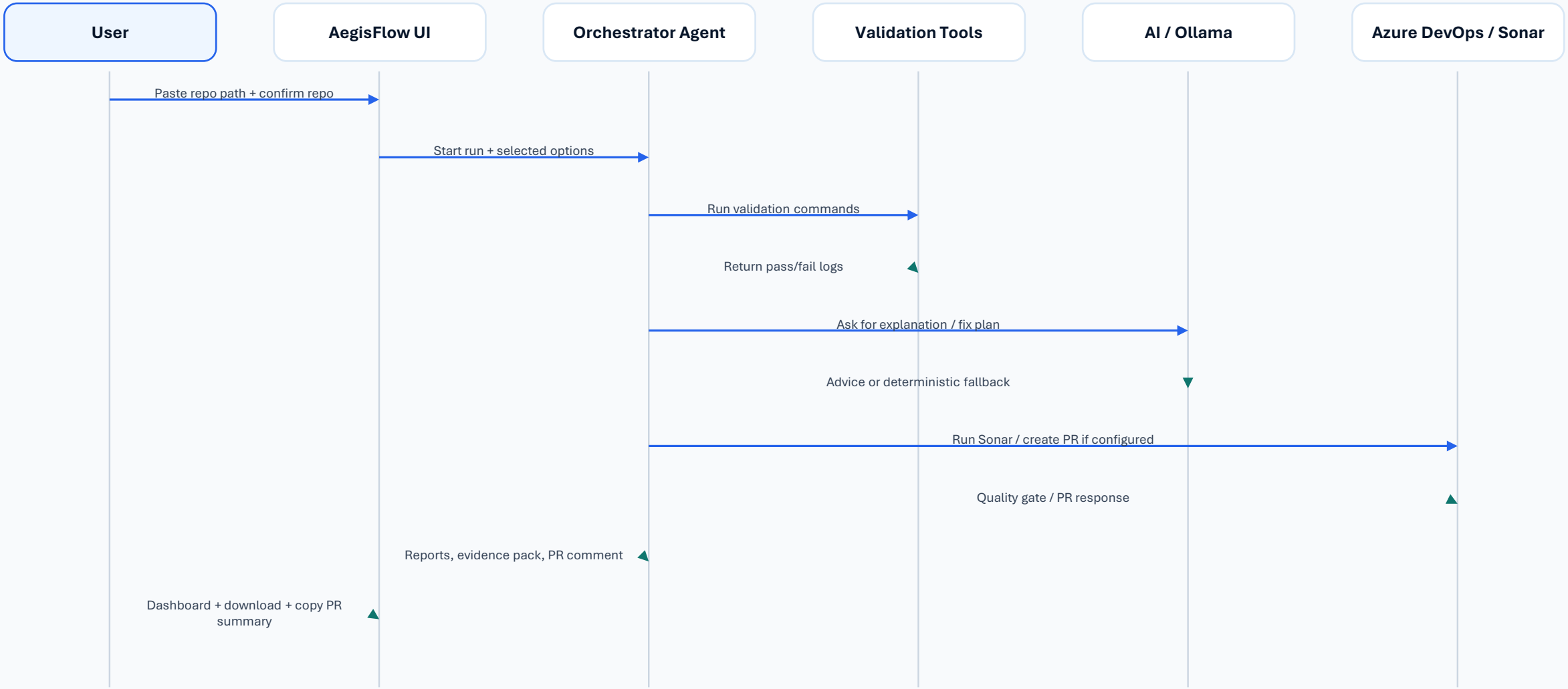
Detailed component architecture

How AegisFlow is composed internally and how data moves between components.



End-to-end sequence diagram

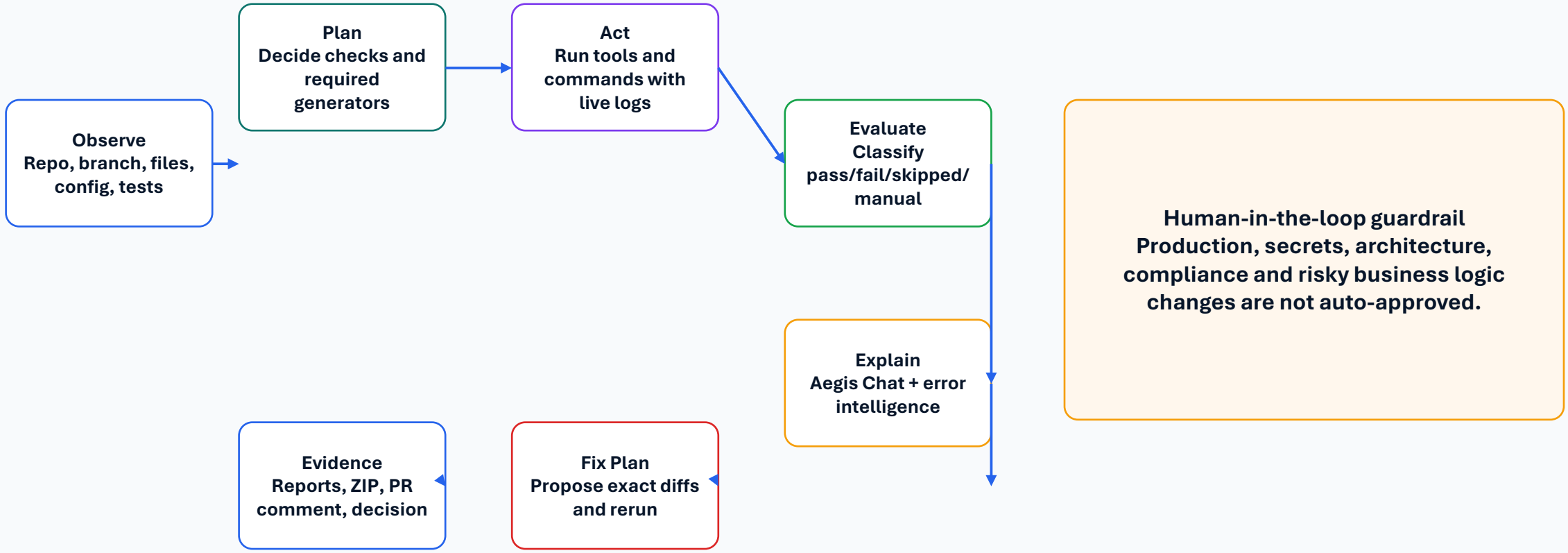
From paste repo path to PR-ready evidence, with safety gates and optional cloud integrations.



Blocking condition: if validations fail, commit/push/PR is blocked unless explicit override is enabled.

Agentic execution loop

Observe → plan → act → evaluate → explain → fix with approval → evidence.



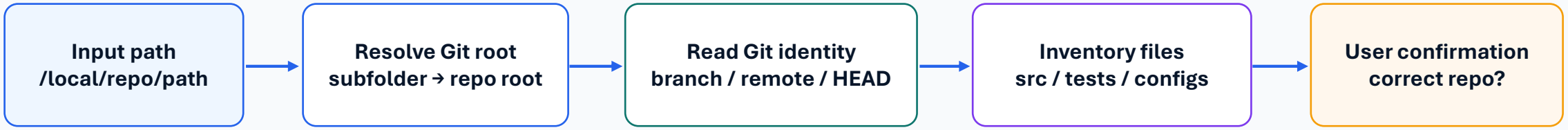
Dashboard modules

What the user sees and why each tab exists.

Tab	What it shows	User value
Repo Preflight	Repo identity, file inventory, source/test mapping	Prevents running on the wrong repo
Acceptance Criteria	Story checklist with pass/fail/manual states	Shows closure readiness
Live Progress	Command timeline, heartbeats, logs, elapsed time	Feels like a real CI/CD run
Report & Downloads	Evidence ZIP, reports, coverage cards, XML previews	Audit-ready evidence
AI Error Intelligence	Failures, severity, owner, suggested fixes	Turns logs into action
AI Fix Plan	Diffs, approval, patch, rerun	Safe agentic correction
Aegis Chat	Ask about Dockerfile, Pytest, Sonar, PR readiness	Interactive explanation
SonarQube	Sonar status, skipped reason, logs, quality gate	Separates coverage from Sonar
Governance / PR	Release decision, sign-off, copyable PR summary	Review-ready communication

Repository preflight and safety checks

AegisFlow first verifies where it is operating before it can modify anything.



Protected folders excluded from all scans and commits

.git, .venv, __pycache__, .pytest_cache, .ruff_cache
orchestrator_reports, .aegisflow_backups, htmlcov
node_modules, dist, build and generated artifacts

Preflight outputs

Repo name, provider, remote URL, current branch
Source files, test files, missing test mapping
CI/CD config files and acceptance criteria status

Acceptance criteria dashboard

The user story checklist becomes a visible pass/fail/manual readiness matrix.

Acceptance area	AegisFlow check	Status logic
Delete unwanted files	Root Dockerfile/.dockerignore, sample files, .vscode, local.settings.json	Pass / fail / manual review
Restructure folders	src/, tests/, src/conf/, function.json, host.json	Pass if expected paths exist
Update imports	Detect legacy module path patterns	Fail if old paths found
Fix parameters.yaml	Detect local absolute paths	Fail/manual if hardcoded local path exists
Split requirements	Prod vs dev dependency files	Pass/partial based on dev tools location
Pipeline files	azure-pipelines.yml, azure-function.config.yml, Sonar config	Pass if present and parseable
Verify locally	Ruff, tests, coverage, Dockerfile lint	Pass if validation events pass
Cloud-only items	Branch policies, deployment, rollback, approvals	Manual/cloud-only until Azure verifies

Validation engine and quality gates

Local checks mirror the cloud PR validation and CI quality gates.

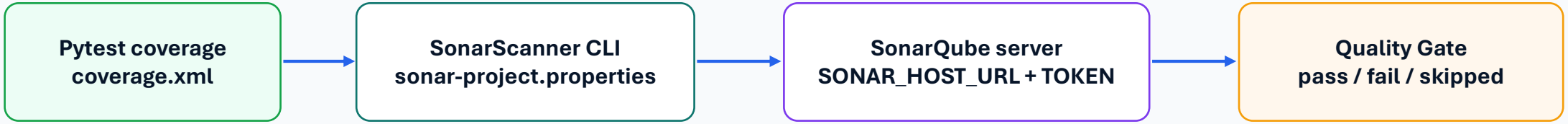
Code quality	Security	Testing + coverage	Container + deps	Static analysis
Python compile Ruff format check Ruff lint check	Lightweight secret scan detect-secrets Bandit SAST	pytest pytest-cov coverage.xml test-results.xml	Hadolint Dockerfile requirements install pip-audit-ready	SonarScanner coverage upload quality gate wait

Each gate emits structured events: status, command, elapsed time, logs, category, severity and suggested owner.

Pass → evidence pack
Fail → owner + fix plan
Skipped → missing credential or cloud-only dependency

SonarQube / SonarCloud integration

AegisFlow distinguishes Azure DevOps coverage from SonarQube quality gate analysis.



Outputs shown in AegisFlow

Sonar passed/failed/skipped card
Scanner command and logs
Missing variable explanation
Quality gate wait result

Outputs shown in SonarQube

Bugs, vulnerabilities, code smells
Duplications and security hotspots
Coverage imported from coverage.xml
Long-term quality history

Azure DevOps Code Coverage tab is separate: PublishCodeCoverageResults renders coverage.xml inside the pipeline run.

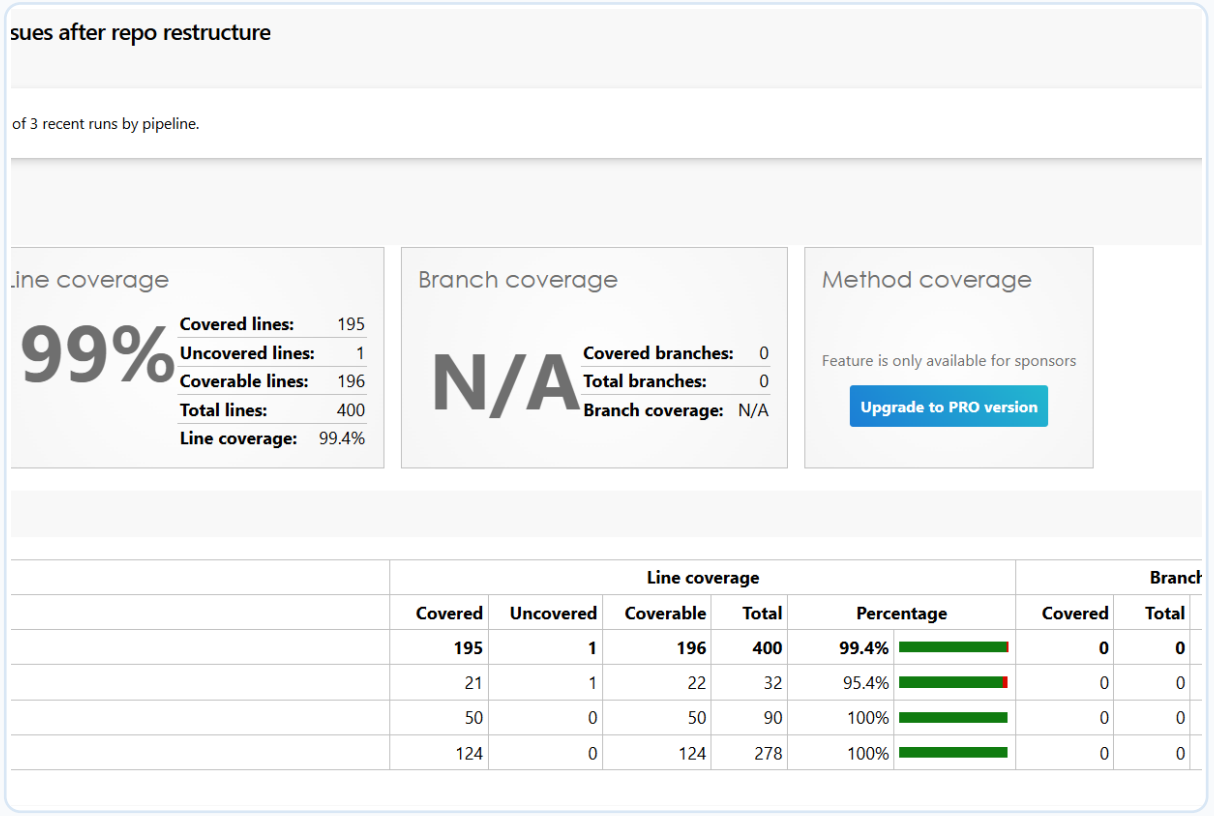
Evidence pack and dashboard reporting

Everything needed for review is visible on the dashboard and downloadable as a ZIP.

Artifact	Purpose	Dashboard preview
Markdown report	Human-readable evidence summary	Preview tab + download
JSON report	Machine-readable run detail	Expandable JSON preview
coverage.xml	Cobertura coverage for Azure DevOps and Sonar	Azure DevOps-style cards and file table
test-results.xml	JUnit test results	Pass/fail counts and raw XML preview
Sonar output	Scanner log and quality gate context	SonarQube tab + report preview
PR comment	Copyable summary for PR review	PR Comment tab
Evidence ZIP	One package for audit/review	ZIP contents table + download

Azure DevOps-style coverage view in AegisFlow

AegisFlow renders coverage.xml in a visual form similar to Azure DevOps Code Coverage.



sues after repo restructure

of 3 recent runs by pipeline.

ine coverage

99%

Covered lines:	195
Uncovered lines:	1
Coverable lines:	196
Total lines:	400
Line coverage:	99.4%

Branch coverage

N/A

Covered branches:	0
Total branches:	0
Branch coverage:	N/A

Method coverage

Feature is only available for sponsors

Upgrade to PRO version

	Line coverage					Branch	
	Covered	Uncovered	Coverable	Total	Percentage	Covered	Total
	195	**1**	**196**	**400**	**99.4%**	**0**	**0**
	21	1	22	32	95.4%	0	0
	50	0	50	90	100%	0	0
	124	0	124	278	100%	0	0

99%
Line coverage

195
Covered lines

1
Uncovered

Data source
pytest-cov → coverage.xml

→

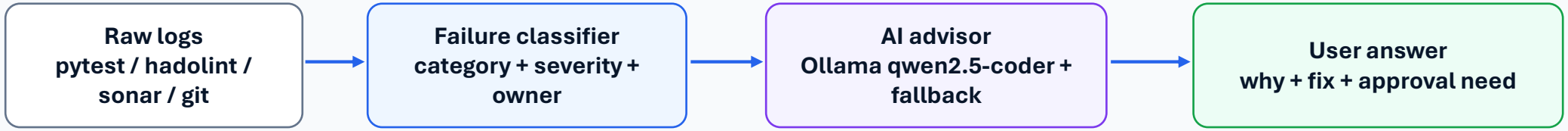
Azure DevOps
PublishCodeCoverageResults

Why this matters

Developers do not need to open Azure DevOps to inspect coverage evidence.
Reviewers see the same coverage concept inside the AegisFlow evidence dashboard.
The same coverage.xml can feed Azure DevOps and SonarQube.

AI Error Intelligence and Aegis Chat

The agent converts failing logs into actionable ownership and next steps.



Failure categories

developer code issue
test expectation issue
tooling/dependency issue
pipeline configuration issue
security/governance issue

Chat examples

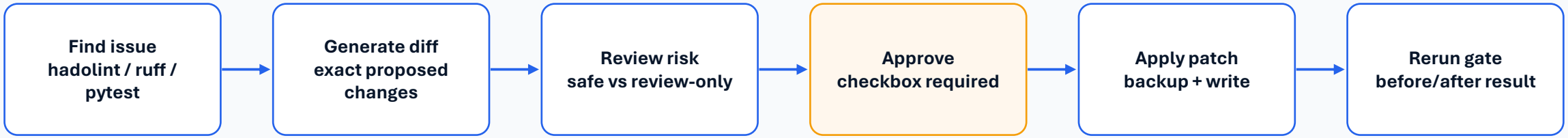
Why did Dockerfile lint fail?
Why did Pytest fail after auto-repair?
Can this be fixed safely?
Is this PR ready for review?

Safety behavior

No blind business logic edits
No secret exposure
No production approval automation
Deterministic fallback if LLM unavailable

AI Fix Plan: review → approve → apply → rerun

Agentic correction is controlled by exact diffs and user approval.



Fix classes

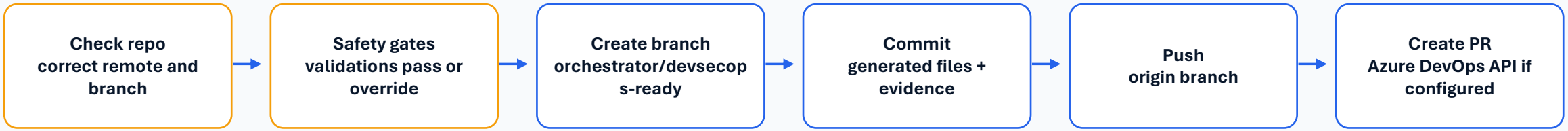
Safe deterministic fixes: formatting, generated tests, cache cleanup, selected Dockerfile policy comments.
Review-only fixes: business logic, dependency version pinning, architecture, secrets, production settings.

Dockerfile specific flow

Run hadolint and parse DL rule numbers.
Create safe diff or mark review-only.
Approve patch, rerun hadolint, show result.

Git and PR automation

AegisFlow can create a branch, commit, push, and prepare a PR when safety gates allow it.



Required configuration

Repo path and user confirmation
Expected repo/remote keyword if needed
AZDO_ORG_URL, AZDO_PROJECT, AZDO_REPO_ID, AZDO_PAT for automatic PR

Safety gate behavior

By default, failed validations block commit/push/PR.
Override is explicit and visible.
Generated folders and backups are protected from commit.

Pipeline mapping: local cockpit ↔ Azure DevOps target state

AegisFlow aligns with the expected PR validation, CI, and CD stages.

Pipeline stage	Expected target-state checks	AegisFlow support
PR validation	Formatting, linting, secret scan, AI PR review, unit tests, SonarQube diff scan, config validation	Local equivalents + PR comment + acceptance criteria
CI	Key Vault fetch, install prod dependencies, pip-audit, Sonar full scan, zip package, publish artifact	Requirements install, Sonar scan, evidence artifact; Key Vault/cloud items marked manual
CD	Download artifact, resolve Azure Function, zip deploy, health check, rollback	Deployment readiness checks; actual deploy/rollback needs Azure credentials/pipeline
Governance	Branch policies, approvals, audit logs, managed identity, no direct prod access	Manual/cloud-only statuses + release decision support

This avoids false positives: AegisFlow marks cloud-only capabilities as manual/partial unless Azure DevOps, SonarQube, or Azure credentials are configured.

Security and governance controls

The system is agentic, but it deliberately preserves human control over sensitive decisions.

Repository hygiene
local.settings.json, .env, generated folders, caches ignored

Secret governance
secret scans, no token commit, Key Vault recommended

Quality governance
Ruff, tests, coverage, Sonar quality gate

Approval governance
PR comment, HITL review, release decision support

Audit evidence
reports, logs, coverage, XML, ZIP, raw command output

Never auto-approve

Production deployment decisions
Secrets/access governance
Architecture approval
Compliance sign-off
Risky business logic changes

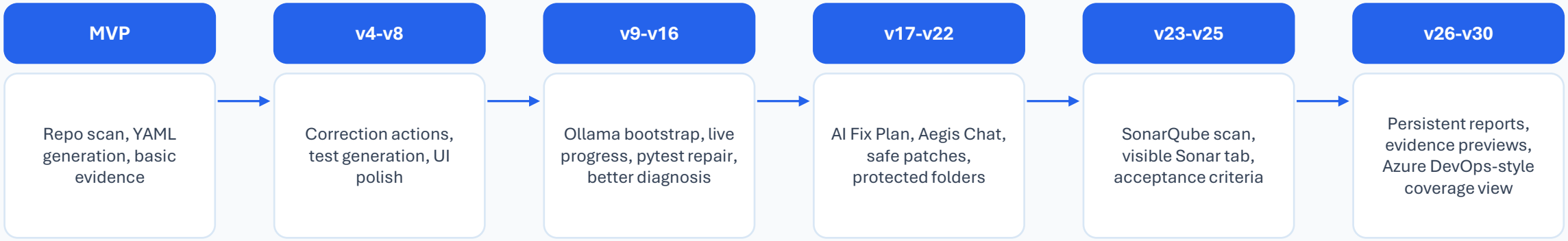
Technology stack and integration surface

AegisFlow combines deterministic DevSecOps tooling with an optional local LLM assistant.

Layer	Technologies	Role
UI	Streamlit	Dashboard, tabs, download buttons, state persistence
Agent runtime	Python, subprocess, session state	Orchestration, command execution, run events
Code quality	compileall, Ruff, Hadolint	Linting, formatting, Dockerfile validation
Security	detect-secrets, Bandit, secret regexes	Secrets and SAST checks
Testing	pytest, pytest-cov, coverage.py, JUnit XML	Unit test and coverage evidence
AI	Ollama, qwen2.5-coder:7b, deterministic fallback	Aegis Chat and recommendations
DevOps	Git, Azure DevOps API, Azure Pipelines YAML	Branch, commit, push, PR, pipeline readiness
Static analysis	SonarScanner, SonarQube/SonarCloud	Coverage upload and quality gate

What was implemented through iterations

The prototype matured from a scanner into a full DevSecOps cockpit.



Current product state

Local-first CI/CD validation cockpit
Acceptance criteria and PR readiness dashboard
Evidence pack and visual coverage view
AI-assisted diagnosis with approved patch workflow

Next enterprise additions

Azure DevOps pipeline history + trigger API
Branch policy verification
Real deployment and health-check monitor
Teams/email notifications and org-wide dashboard

Enterprise roadmap

What remains for a production-grade platform rollout.

Phase	Capability	Description
1	Azure DevOps connected mode	Fetch pipeline runs, trigger validation, monitor logs, attach PR comments via API
2	Central template compliance	Validate repos against central pipeline template and version
3	Cloud deployment validation	Resolve function app, run health check, support rollback visibility
4	Org governance dashboard	Trend coverage, failures, Sonar gate, security issues across repositories
5	Policy-as-code	Custom rules for repo structure, approvals, secrets, dependency gates and compliance

Target vision: AegisFlow becomes the developer-facing UI orchestrator for DevSecOps, while Azure DevOps remains the source of truth for protected branches, cloud pipeline execution, and production deployment.

How to explain AegisFlow to the team

A concise technical narrative for demos and stakeholder updates.

AegisFlow is an agentic DevSecOps cockpit that takes a local Azure Function repo and validates it against the CI/CD acceptance criteria before PR.

1	**It reads repo identity first**	Git root, branch, remote, changed files, and source/test structure.
2	**It runs the same gates developers care about**	Ruff, secrets, Bandit, tests, coverage, Dockerfile, SonarQube.
3	**It explains failures**	Who owns it, why it failed, how to fix it, and whether it is safe to patch.
4	**It creates evidence**	ZIP, Markdown, JSON, coverage.xml, test-results.xml and PR comment.
5	**It preserves governance**	Production, secrets, architecture and compliance approvals remain human-owned.